

● PYTHON BASICS

CONCEPT	EXAMPLE
Hello World	<code>print("Hello, World!")</code>
Comment	<code># single line</code> <code>"""multi-line"""</code>
Variable	<code>x = 10; name = "py"</code>
Multi-assign	<code>a, b, c = 1, 2, 3</code>
Swap	<code>a, b = b, a</code>
Dynamic type	<code>x = 5; x = "hi" # OK</code>
Type check	<code>type(x) isinstance(x, int)</code>
Input	<code>n = input("Enter: ")</code>

NAMING CONVENTIONS

snake_case	variables, functions
PascalCase	classes
UPPER_CASE	constants
_private	internal use
__dunder__	magic methods

STRING FORMATTING

```
# f-string (fastest)
f"Hello {name}, age={age:.2f}"
# .format()
"{ } is {}".format(name, age)
# % formatting
"%s is %d" % (name, age)
```

● STRINGS

<code>s = "Hello, World!"</code> <code>s[0] # 'H'</code> <code>s[-1] # '!'</code> <code>s[1:5] # 'ello'</code> <code>s[::-1] # reversed</code> <code>s[0::2] # every 2nd</code>	
METHOD	RESULT
<code>.lower()</code>	"hello, world!"
<code>.upper()</code>	"HELLO, WORLD!"
<code>.strip()</code>	removes whitespace
<code>.replace(a,b)</code>	swap substrings
<code>.split(",")</code>	→ list
<code>",".join(lst)</code>	list → str
<code>.find("o")</code>	index or -1
<code>.count("l")</code>	occurrences
<code>.startswith("H")</code>	True/False
<code>.endswith("!")</code>	True/False
<code>.title()</code>	Title Case
<code>.zfill(5)</code>	"00042"
<code>.center(10)</code>	center pad
<code>.isdigit()</code>	all digits?
<code>.isalpha()</code>	all alpha?
<code># Multiline</code> <code>s = ""line1</code> <code>line2""</code> <code># Raw string (no escape)</code> <code>r = r"\n not newline"</code>	

● NUMBERS & OPERATORS

NUMBER TYPES	ARITHMETIC
<code>int</code> <code>x = 42</code>	<code>+</code> <code>-</code> <code>*</code> add/sub/mul
<code>float</code> <code>x = 3.14</code>	<code>/</code> float div
<code>complex</code> <code>x = 2+3j</code>	<code>//</code> floor div
<code>bool</code> <code>True / False</code>	<code>%</code> modulo
CONVERSIONS	
<code>int("42")</code> <code>float(3)</code>	BITWISE
<code>str(42)</code> <code>bool(0)</code>	<code>&</code> <code> </code> <code>^</code> AND/OR/XOR
	<code>~</code> NOT
	<code><<</code> <code>>></code> shift
COMPARISON & LOGICAL	
<code>=</code> <code>≠</code> <code>></code> <code><</code> <code>≥</code> <code>≤</code>	<i># comparison</i>
<code>and</code> <code>or</code> <code>not</code>	<i># logical</i>
<code>in</code> <code>not in</code>	<i># membership</i>
<code>is</code> <code>is not</code>	<i># identity</i>
<i># Chained comparison</i>	
<code>1 < x < 10</code>	<i># Pythonic!</i>
ASSIGNMENT	
<code>x += 1</code> <code>x -= 1</code> <code>x *= 2</code>	
<code>x /= 2</code> <code>x /= 2</code> <code>x %= 3</code>	
<code>x **= 2</code> <code>x /= 1</code> <code>x = 1</code>	
<code>x := expr</code> <i># walrus op</i>	
MATH BUILT-INS	
<code>abs(-5)=5</code> <code>round(3.14,1)=3.1</code>	
<code>pow(2,10)=1024</code> <code>divmod(9,4)=(2,1)</code>	

● CONTROL FLOW & LOOPS

IF / ELIF / ELSE
<code>if x > 0:</code> <code>print("pos")</code> <code>elif x == 0:</code> <code>print("zero")</code> <code>else:</code> <code>print("neg")</code>
TERNARY
<code>val = "yes" if cond else "no"</code>
MATCH-CASE (3.10+)
<code>match cmd:</code> <code>case "quit": exit()</code> <code>case "go": move()</code> <code>case _: print("?")</code>
FOR / WHILE
<code>for i in range(5): ...</code> <i># 0-4</i> <code>for i in range(2,10,2): ...</code> <i># 2,4,6,8</i> <code>for i, v in enumerate(lst): ...</code> <code>for a, b in zip(l1, l2): ...</code> <code>while cond:</code> <code>if done: break</code> <code>if skip: continue</code> <code>pass</code> <i># no-op placeholder</i> <code>else:</code> <i># runs if no break</i> <code>...</code>
COMPREHENSIONS
<code>[x**2 for x in range(5)]</code> <i># list</i> <code>[x for x in lst if x > 0]</code> <i># filter</i> <code>{k:v for k,v in d.items()}</code> <i># dict</i> <code>{x**2 for x in lst}</code> <i># set</i> <code>(x**2 for x in lst)</code> <i># gen</i> <code>[[r*c for c in row] for row in m]</code> <i># nested</i>

● LISTS · TUPLES · SETS

LISTS	TUPLES (IMMUTABLE)	SETS
<code>lst = [1, 2, 3]</code> <code>lst[0] # 1</code> <code>lst[-1] # 3</code> <code>lst[1:3] # [2,3]</code> <code>lst[::-1] # reversed</code>	<code>t = (1, 2, 3)</code> <code>t = 1, 2, 3</code> <i># packing</i> <code>a, b, c = t</code> <i># unpack</i> <code>a, *b = t</code> <i># star unpack</i> <code>t[0] # index</code> <code>t.count(1) # → 1</code> <code>t.index(2) # → 1</code> <code>tuple(lst) # from list</code>	<code>s = {1, 2, 3}</code> <code>s = set()</code> <i># empty</i> <code>s.add(4)</code> <code>s.update([5,6])</code> <code>s.remove(1)</code> <i># KeyError if miss</i> <code>s.discard(1)</code> <i># safe</i> <code>s.pop()</code> <i># random item</i>
METHOD	ACTION	OP
<code>.append(x)</code>	add end	SYNTAX
<code>.extend(it)</code>	add iterable	<code>union</code> <code>a b</code>
<code>.insert(i,x)</code>	insert at i	<code>intersect</code> <code>a & b</code>
<code>.remove(x)</code>	remove first x	<code>difference</code> <code>a - b</code>
<code>.pop(i)</code>	remove & return	<code>sym diff</code> <code>a ^ b</code>
<code>.sort()</code>	in-place sort	<code>subset</code> <code>a ≤ b</code>
<code>.reverse()</code>	in-place rev	<code>superset</code> <code>a ≥ b</code>
<code>.copy()</code>	shallow copy	<code>frozenset([1,2])</code> <i># immutable</i>
<code>.index(x)</code>	find index	
<code>.count(x)</code>	occurrences	
<code>.clear()</code>	empty list	
<code>lst + [4]</code> <i># concat</i> <code>lst * 3</code> <i># repeat</i> <code>len(lst)</code> <i># length</i> <code>2 in lst</code> <i># True</i>	NAMED TUPLE <code>from collections import namedtuple</code> <code>P = namedtuple('P', ['x','y'])</code> <code>p = P(1, 2); p.x</code>	
	SORTING <code>sorted(lst)</code> <i># new</i> <code>sorted(lst, reverse=True)</code> <code>sorted(lst, key=lambda x:x[1])</code> <code>lst.sort(key=abs)</code>	

● DICTIONARIES

<code>d = {"a":1, "b":2}</code> <code>d = dict(a=1, b=2)</code> <code>d["a"]</code> <i># → 1</i> <code>d.get("z", 0)</code> <i># safe get</i> <code>d["c"] = 3</code> <i># add/update</i> <code>d.pop("a")</code> <i># remove</i> <code>d.update({"x":9})</code> <i># merge in</i> <code>d1 d2</code> <i># merge (3.9+)</i>	
METHOD	RETURNS
<code>.keys()</code>	dict_keys view
<code>.values()</code>	dict_values view
<code>.items()</code>	(k,v) pairs
<code>.setdefault(k,v)</code>	get or set
<code>.clear()</code>	empty dict
<code>.copy()</code>	shallow copy
ITERATION	
<code>for k in d: ...</code> <code>for k, v in d.items(): ...</code> <i># defaultdict</i> <code>from collections import defaultdict</code> <code>dd = defaultdict(list)</code> <code>dd["a"].append(1)</code>	

● FUNCTIONS

```
def greet(name: str, age: int = 0) → str:
    """Docstring here."""
    return f"Hi {name}"

# Positional + keyword args
greet("Alice", age=30)

# *args – tuple of extras
def f(*args): ...

# **kwargs – dict of extras
def f(**kwargs): ...

# Combined
def f(a, b=1, *args, **kw): ...
```

LAMBDA

```
sq = lambda x: x**2
add = lambda a, b: a + b
sorted(lst, key=lambda x: x[1])
```

RECURSION

```
def fact(n):
    if n ≤ 1: return 1
    return n * fact(n-1)
```

DECORATORS

```
from functools import wraps
def timer(func):
    @wraps(func)
    def wrapper(*a, **kw):
        import time
        t = time.time()
        r = func(*a, **kw)
        print(time.time()-t)
        return r
    return wrapper
```

@timer

```
def slow(): ...
```

FUNCTIONAL TOOLS

```
map(str, [1,2,3])           # lazy map
filter(lambda x:x>0, lst)
from functools import reduce
reduce(lambda a,b:a+b, [1,2,3]) #6
# Apply & expand
list(map(lambda x:x*2, lst))
```

ITERATORS & GENERATORS

```
it = iter([1,2,3])
next(it)           # → 1
def gen(n):
    for i in range(n):
        yield i * 2
g = gen(5)
next(g)            # → 0
```

● EXCEPTION HANDLING

```
try:
    x = 1 / 0
except ZeroDivisionError as e:
    print(e)
except (TypeError, ValueError):
    print("type/val err")
else:
    print("no error") # if no exc
finally:
    print("always runs")
```

RAISE & ASSERT

```
raise ValueError("msg")
raise           # re-raise
assert x > 0, "must be pos"
```

CUSTOM EXCEPTION

```
class MyErr(Exception):
    def __init__(self, msg):
        super().__init__(msg)
```

COMMON EXCEPTIONS

ValueError	bad value
TypeError	wrong type
KeyError	missing key
IndexError	out of range
AttributeError	no attribute
FileNotFoundError	missing file
StopIteration	iter ended
RecursionError	stack overflow
MemoryError	out of memory

● OBJECT-ORIENTED PROGRAMMING

```
class Animal:
    species = "unknown" # class var

    def __init__(self, name):
        self.name = name # instance var

    def speak(self): # method
        return "..."

    @classmethod
    def create(cls, name):
        return cls(name)

    @staticmethod
    def info():
        return "Animal class"

    def __str__(self):
        return self.name

    def __repr__(self):
        return f"Animal({self.name!r})"

    def __len__(self):
        return len(self.name)
```

```
a = Animal("Cat")
print(a.name) # Cat
```

ENCAPSULATION

```
self.pub   = "public"
self._prot = "protected"
self.__priv= "private" # name-mangled
```

INHERITANCE

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

    def speak(self): # override
        return "Woof!"
```

```
d = Dog("Rex", "Lab")
isinstance(d, Animal) # True
```

SPECIAL (DUNDER) METHODS

__init__	constructor
__str__	str() / print()
__repr__	repr()
__len__	len()
__eq__	==
__lt__	<
__add__	+
__getitem__	obj[i]
__iter__	for loop
__next__	next()
__enter__	with stmt
__exit__	with end
__call__	obj()
__del__	destructor

DATACLASS (3.7+)

```
from dataclasses import dataclass
@dataclass
class Point:
    x: float; y: float
    z: float = 0.0
p = Point(1.0, 2.0)
```

● FILE HANDLING

```
# Best practice: with statement
with open("file.txt", "r") as f:
    content = f.read() # all
    line    = f.readline() # one
    lines   = f.readlines() # list
```

```
with open("out.txt", "w") as f:
    f.write("hello\n")
```

```
with open("out.txt", "a") as f:
    f.write("append me")
```

MODE	MEANING
"r"	read (default)
"w"	write (overwrite)
"a"	append
"x"	create (fail if exists)
"rb"	read binary
"wb"	write binary
"r+"	read & write

PATHLIB

```
from pathlib import Path
p = Path("dir/file.txt")
p.read_text()
p.write_text("hi")
p.exists() p.is_file()
p.parent  p.suffix
list(p.iterdir())
p / "sub" / "file.py"
```

● TYPE HINTS

```
from typing import (
    Optional, Union, Any,
    List, Dict, Tuple,
    Callable, Iterator
)
# Variable annotations
x: int = 5
name: str
lst: list[int] = []

# Function signature
def fn(
    a: int,
    b: Optional[str] = None,
    *args: float
) → list[str]: ...
```

HINT	MEANING
int str	Union (3.10+)
Optional[X]	X None
Any	any type
list[int]	list of ints
dict[str,int]	str→int dict
tuple[int, ...]	variable tuple
Callable[[int],str]	func type

● MODULES & PACKAGES

```
import math           # whole module
from math import sqrt, pi  # specific
import numpy as np      # alias
from . import sibling     # relative

# Entry point guard
if __name__ == "__main__":
    main()
```

PIP & VENV

```
pip install package
pip install -r requirements.txt
pip freeze > requirements.txt
```

```
python -m venv .venv
.venv\Scripts\activate    # Win
source .venv/bin/activate # Unix
deactivate
```

● MATH & RANDOM

MATH

math.sqrt(x)	$\sqrt{x}$
math.ceil(x)	$\lceil x \rceil$
math.floor(x)	$\lfloor x \rfloor$
math.log(x,b)	log base b
math.log2(x)	$\log_2$
math.sin/cos/tan	trig
math.pi	3.14159...
math.e	2.71828...
math.inf	∞
math.factorial(n)	n!
math.gcd(a,b)	GCD
math.comb(n,k)	C(n,k)
math.isclose(a,b)	$\approx$ equal?

RANDOM

random.random()	[0.0, 1.0)
random.randint(a,b)	[a, b]
random.choice(lst)	one item
random.choices(lst,k=3)	k items
random.shuffle(lst)	in-place
random.sample(lst,k)	no replace
random.seed(42)	reproducible

● DATETIME & TIME

```
from datetime import datetime, date, timedelta
import time

datetime.now()           local now
datetime.utcnow()        UTC now
date.today()             today's date
timedelta(days=7)        duration
dt.strptime("%Y-%m-%d")  format
datetime.strptime(s,fmt) parse
dt.timestamp()           Unix ts
time.time()              epoch float
time.sleep(1.5)          pause 1.5s
```

FORMAT CODES			
%Y	2025	%m	06
%d	13	%H	14
%M	30	%S	59
%A	Friday	%B	June

● OS · SYS · PATHLIB

OS

os.getcwd()	current dir
os.chdir(path)	change dir
os.listdir(path)	list dir
os.makedirs(p,exist_ok=T)	mkdir -p
os.remove(file)	delete file
os.rename(src,dst)	rename
os.path.join(a,b)	safe join
os.path.exists(p)	path exists?
os.path.basename(p)	filename
os.environ.get("K")	env var
os.walk(path)	dir tree

SYS

sys.argv	CLI args list
sys.exit(0)	exit code
sys.path	module paths
sys.version	Python ver
sys.stdin/out/err	streams

● COLLECTIONS · ITERTOOLS

COLLECTIONS		ITERTOOLS	
CLASS	USE CASE	FUNCTION	DESCRIPTION
Counter	count occurrences	chain(a,b)	concat iterables
defaultdict	default factory	islice(it,n)	slice iterator
OrderedDict	insertion order	product(a,b)	cartesian product
namedtuple	tuple with names	permutations(it,r)	ordered r-subsets
deque	fast both-end ops	combinations(it,r)	unordered r-subsets
ChainMap	combine dicts	groupby(it,key)	group consecutive
<pre>from collections import Counter c = Counter("abracadabra") c.most_common(2) # [('a',5),('b',2)] c['a'] # 5 c + Counter("aab") # add counts from collections import deque dq = deque([1,2,3], maxlen=5) dq.appendleft(0) dq.popleft() dq.rotate(1)</pre>		count(start,step)	infinite counter
		cycle(it)	repeat forever
		repeat(val,n)	repeat n times
		accumulate(it,op)	running totals
		zip_longest(a,b)	zip with fill
		starmap(fn, it)	map with unpack
		takewhile(pred,it)	take while true
		dropwhile(pred,it)	drop while true
		flatten	chain.from_iterable
		<pre>from itertools import combinations list(combinations("ABC", 2)) # [('A','B'),('A','C'),('B','C')] from itertools import product list(product(range(2), repeat=3)) # all 3-bit combos</pre>	

● RE (REGEX)

FUNCTION		ACTION	
re.search(p,s)		first match anywhere	
re.match(p,s)		match at start	
re.findall(p,s)		all matches list	
re.finditer(p,s)		iter of matches	
re.sub(p,r,s)		replace matches	
re.split(p,s)		split by pattern	
re.compile(p)		compiled pattern	
PATTERNS			
.	any char	\d	digit
\w	word char	\s	whitespace
^	start	\$	end
*	0+	+	1+
?	0 or 1	{n}	exactly n
[abc]	char set	[^a]	not a
(ab)	group	a b	or
<pre>import re m = re.search(r'\d+', "abc123") m.group() # '123' emails = re.findall(r'\S+@\S+', txt) re.sub(r'\s+', ' ', s) # collapse spaces</pre>			

● JSON · CSV

JSON

```
import json
# Encode
s = json.dumps(obj, indent=2)
# Decode
obj = json.loads(s)
# File write
with open("f.json","w") as f:
    json.dump(obj, f, indent=2)
# File read
with open("f.json") as f:
    obj = json.load(f)
```

CSV

```
import csv
with open("f.csv") as f:
    r = csv.reader(f)
    for row in r: ...

with open("f.csv") as f:
    dr = csv.DictReader(f)
    for row in dr:
        print(row["col"])

with open("out.csv","w") as f:
    w = csv.writer(f)
    w.writerows(data)
```

● BUILT-IN FUNCTIONS

FUNCTION	DESCRIPTION	FUNCTION	DESCRIPTION
<code>print(*args, sep, end)</code>	output to stdout	<code>abs(x)</code>	absolute value
<code>input(prompt)</code>	read line from stdin	<code>round(x, n)</code>	round to n places
<code>len(obj)</code>	length of sequence	<code>pow(x, y, mod)</code>	$x*y \% mod$
<code>type(obj)</code>	object's type	<code>divmod(a, b)</code>	(q, r) tuple
<code>isinstance(obj, T)</code>	type check	<code>id(obj)</code>	memory address
<code>range(start, stop, step)</code>	int range	<code>hash(obj)</code>	hash value
<code>enumerate(it, start=0)</code>	(i, val) pairs	<code>dir(obj)</code>	list attributes
<code>zip(*its)</code>	parallel iterate	<code>help(obj)</code>	docstring help
<code>map(fn, it)</code>	apply fn to each	<code>vars(obj)</code>	<code>__dict__</code>
<code>filter(fn, it)</code>	keep if fn True	<code>repr(obj)</code>	repr string
<code>sorted(it, key, rev)</code>	new sorted list	<code>str/int/float/bool(x)</code>	type cast
<code>sum(it, start=0)</code>	total	<code>list/tuple/set/dict(x)</code>	convert
<code>min(it, key)</code>	minimum	<code>chr(n)</code>	int $\rightarrow$ char
<code>max(it, key)</code>	maximum	<code>ord(c)</code>	char $\rightarrow$ int
<code>any(it)</code>	any True?	<code>bin/oct/hex(n)</code>	base convert
<code>all(it)</code>	all True?	<code>open(path, mode)</code>	file handle
		<code>callable(obj)</code>	is callable?
		<code>getattr/setattr(obj,n)</code>	dynamic attr
		<code>hasattr(obj, name)</code>	attr exists?
		<code>eval/exec(str)</code>	run code str

● PYTHON TRICKS & SHORTCUTS

TRICK	CODE	TRICK	CODE
Swap	<code>a, b = b, a</code>	Zip enumerate	<code>for i,(a,b) in enumerate(zip(l1,l2))</code>
Unpack *rest	<code>a, *b, c = lst</code>	Reverse iter	<code>reversed(lst)</code>
Chained cmp	<code>0 < x < 10</code>	Safe index	<code>lst[idx] if idx<len(lst) else None</code>
Walrus	<code>if n:= len(a): ...</code>	All equal	<code>len(set(lst)) == 1</code>
Dict merge	<code>{**d1, **d2}</code>	Max by key	<code>max(d, key=d.get)</code>
Dict merge 3.9	<code>d1 d2</code>	Transpose	<code>list(zip(*matrix))</code>
Underscore num	<code>1_000_000</code>	Chunk list	<code>[lst[i:i+n] for i in range(0,len(lst),n)]</code>
Bool as int	<code>True+True == 2</code>	Dict from 2 lists	<code>dict(zip(keys, vals))</code>
Repeat string	<code>"ha"*3 $\rightarrow$ "hahaha"</code>	Count items	<code>Counter(lst).most_common()</code>
Flatten list	<code>[x for l in ll for x in l]</code>	Flatten dict	<code>{k:v for d in lst for k,v in d.items()}</code>
Remove dups	<code>list(set(lst))</code>	String join	<code>", ".join(map(str, lst))</code>
Ternary chain	<code>a if p1 else b if p2 else c</code>	Context mgr	<code>with A() as a, B() as b: ...</code>

● BIG-O COMPLEXITY REFERENCE

LIST OPERATIONS			DICT & SET OPERATIONS			DEQUE OPERATIONS					
OP	AVG	WORST	OP	DICT	SET						
index <code>l[i]</code>	<code>O(1)</code>	<code>O(1)</code>	lookup	<code>O(1)</code>	<code>O(1)</code>	<code>appendleft</code>	<code>O(1)</code>				
<code>append</code>	<code>O(1)</code>	<code>O(1)</code>	insert	<code>O(1)</code>	<code>O(1)</code>	<code>popleft</code>	<code>O(1)</code>				
<code>pop (end)</code>	<code>O(1)</code>	<code>O(1)</code>	delete	<code>O(1)</code>	<code>O(1)</code>	<code>append/pop</code>	<code>O(1)</code>				
<code>pop (front)</code>	<code>O(n)</code>	<code>O(n)</code>	in	<code>O(1)</code>	<code>O(1)</code>	<code>in</code>	<code>O(n)</code>				
insert	<code>O(n)</code>	<code>O(n)</code>	iteration	<code>O(n)</code>	<code>O(n)</code>	HEAPQ					
remove	<code>O(n)</code>	<code>O(n)</code>	<pre>import heapq h = [3,1,4] heapq.heapify(h) <i>O(n)</i> heapq.heappush(h, x) <i>O(log n)</i> heapq.heappop(h) <i>O(log n)</i> heapq.nlargest(3,lst) <i>O(n log k)</i> heapq.nsmallest(3,l) <i># Max-heap: negate values!</i> heapq.heappush(h,-x)</pre>								
in (search)	<code>O(n)</code>	<code>O(n)</code>						SORTING ALGOS			
sort	<code>O(n log n)</code>	<code>O(n log n)</code>						ALGO	BEST	AVG	WORST
slice <code>a:b</code>	<code>O(k)</code>	<code>O(k)</code>						Timsort*	<code>O(n)</code>	<code>O(n log n)</code>	<code>O(n log n)</code>
reverse	<code>O(n)</code>	<code>O(n)</code>						Quick	<code>O(n log n)</code>	<code>O(n log n)</code>	<code>O(n²)</code>
len	<code>O(1)</code>	<code>O(1)</code>	Merge	<code>O(n log n)</code>	<code>O(n log n)</code>	<code>O(n log n)</code>	BISECT (BINARY SEARCH) <pre>import bisect bisect.bisect_left(a,x) <i>O(log n)</i> bisect.insort(a,x) <i>O(n)</i></pre>				
extend	<code>O(k)</code>	<code>O(k)</code>	Heap	<code>O(n log n)</code>	<code>O(n log n)</code>	<code>O(n log n)</code>					
			Bubble	<code>O(n²)</code>	<code>O(n²)</code>	<code>O(n²)</code>					
* Python's built-in sort											
SEARCH											
Linear search				<code>O(n)</code>							
Binary search				<code>O(log n)</code>							
Hash table				<code>O(1)</code>	avg						
BST lookup				<code>O(log n)</code>	avg						

● FUNCTOOLS · CONTEXT MANAGERS

FUNCTOOLS	
<code>@lru_cache(maxsize=128)</code>	memoize
<code>@cache</code>	unbounded cache (3.9+)
<code>partial(fn, arg)</code>	fix arguments
<code>reduce(fn, it)</code>	fold left
<code>@total_ordering</code>	fill comparison ops
<code>@wraps(fn)</code>	preserve metadata
<pre>from functools import lru_cache @lru_cache(maxsize=None) def fib(n): if n < 2: return n return fib(n-1) + fib(n-2)</pre>	
CUSTOM CONTEXT MANAGER	
<pre>from contextlib import contextmanager @contextmanager def managed(): print("enter") try: yield finally: print("exit") with managed(): ...</pre>	
<pre># suppress exceptions from contextlib import suppress with suppress(FileNotFoundError): os.remove("x.txt")</pre>	

● TIPS · GOTCHAS · BEST PRACTICES

GOTCHA	FIX
Mutable default arg	<code>def f(lst=None): lst = lst or []</code>
Late binding closure	<code>lambda i=i: i</code>
<code>==</code> vs <code>is</code>	use <code>is</code> only for None/True/False
Shallow copy	<code>import copy; copy.deepcopy(obj)</code>
Float precision	<code>from decimal import Decimal</code>
Global var modify	<code>global x</code> inside function
STRING INTERNING & IDENTITY	
<pre>a = "hello"; b = "hello" a is b # True (interned) a = "hello world" b = "hello world" a is b # False (not interned)</pre>	
USEFUL IDIOMS	
<pre># Dict comprehension from zip d = {k:v for k,v in zip(keys,vals)} # Group by with defaultdict grp = defaultdict(list) for item in items: grp[item.key].append(item) # One-line file read lines = Path("f.txt").read_text().splitlines()</pre>	
PERFORMANCE TIPS	
Use <code>set/dict</code> for lookups	$O(1)$ vs $O(n)$
Use generators for big data	memory efficient
Use <code>join</code> not <code>+</code> for strs	avoids copies
Use <code>enumerate</code> not <code>range+index</code>	cleaner
Local vars faster in loops	avoid global lookup